Requirements :

- User can add a description for each task.
- User can assign a category (Work, Study, Personal, etc.).
- User can set recurring tasks (Daily, Weekly, Monthly).
- User can set multiple reminders for a task.
- User can set reminder intervals (5, 10, 15, 30 minutes before).
- User can filter tasks by priority, date, category, or status.
- User can view tasks due today.
- User can view overdue tasks.
- User can export tasks to a CSV file.
- User can import tasks from a CSV file.
- System automatically saves tasks to prevent data loss.
- System validates date and time entered by the user.
- System prevents duplicate task IDs.
- User can clear completed tasks.
- User can restore deleted tasks from a recycle bin (optional).
- User can view task statistics (Total, Completed, Pending, Overdue).
- User can archive completed tasks.
- User can add notes to tasks.
- User can choose notification sound or silent reminders.
- System displays a confirmation message before deleting a task.
- User can exit the application safely after saving all data.
- System records the task creation and completion timestamps.
- User can edit only specific fields without re-entering all task details.
- System automatically sorts upcoming tasks on startup.

Automatic Task Scheduler – Implementation Plan

1. Create New Task

- Create a Task class.
- Take task details (title, description, date, time, priority) from the user.
- Store the task in a list and save it to a CSV file.

2. Set Task Priority

- Ask the user to select High, Medium, or Low.
- Save the selected priority with the task.

3. Assign Date and Time

- Accept date and time input.
- Validate the format before saving.

4. View All Tasks

- Read all tasks from the CSV file.
- Display them in a formatted table.

5. Search Task

- Ask for the task ID or title.
- Search the task list and display the matching task.

6. Update Task Details

- Search the task.
- Allow the user to edit the required fields.
- Save the updated data back to the CSV file.

7. Delete Task

- Search the task by ID.
- Remove it from the task list.
- Save the updated list.

8. Mark Task as Completed

- Change the task status from "Pending" to "Completed".
- Save the updated status.

9. Display Task Reminders

- Continuously check the current date and time.
- If they match a task's schedule and the task is pending, display a reminder message.

10. Maintain Task History

task is completed, keep it in the history instead of deleting it.

11. Sort Tasks

- Use Python's sort() function.
- Sort by date, time, or priority.

12. View Completed and Pending Tasks

- Filter tasks based on their status.
- Display only Completed or only Pending tasks.

13. Add Task Description

- Accept a description while creating or updating a task.
- Save it with the task details.

14. Task Categories

- Let the user choose a category such as Work, Study, Personal, or Others.
- Save it with the task.

15. Recurring Tasks

- Store a repeat option (Daily, Weekly, Monthly).
- After completion, automatically generate the next occurrence.

16. Multiple Reminders

- Allow the user to specify multiple reminder times.
- Trigger notifications at each scheduled reminder.

17. Reminder Before Due Time

- Calculate reminder time (for example, 10 minutes before the due time).
- Notify the user before the task is due.

18. Filter Tasks

- Filter the task list by date, priority, category, or status.
- Display only matching tasks.

19. View Today's Tasks

- Compare each task's date with today's date.
- Display today's tasks only.

20. View Overdue Tasks

- Compare the current date and time with each task.

- Show tasks whose deadline has passed and are still pending.

21. Export Tasks

- Write all tasks to a CSV file.
- Allow the user to save a backup copy.

22. Import Tasks

- Read task data from a CSV file.
- Load it into the application.

23. Automatic Data Saving

- Save the CSV file automatically after every add, update, delete, or complete operation.

24. Task Statistics

- Count total, completed, pending, and overdue tasks.
- Display the counts to the user.

25. Backup and Restore Data

- Create a backup copy of the task file.
- Restore the backup whenever the user requests it.

The Module dev are use:

```
csv         # Save and read tasks
datetime    # Date and time handling
time        # Reminder checking
os          # File operations
shutil      # Backup and restore
```

How the Automatic Task Scheduler Works (Without GUI)

1. The user runs the Python program from the terminal or command prompt.
2. The system displays a menu with different options.
3. The user enters the number of the desired operation.
4. The system accepts the required input from the user.
5. The program validates the entered data.
6. The task is saved in a CSV file.
7. The system automatically updates the CSV file after every change.
8. The user can view all saved tasks.
9. The user can search for a task using its ID or title.
10. The user can update task details.
11. The user can delete unwanted tasks.
12. The user can mark tasks as completed.
13. The system stores completed tasks in the task history.
14. The user can view completed and pending tasks separately.
15. The user can sort tasks by date, time, or priority.
16. The user can filter tasks based on category, status, or priority.
17. The system continuously checks the current date and time.
18. When the scheduled time matches, the system displays a reminder notification.
19. The user can view today's tasks and overdue tasks.
20. The user can export, import, back up, and restore task data.
21. The user exits the application, and all task data remains saved for the next time the program is opened.

Developer Implementation Requirements

1. Create a Task class to store task information.
2. Create a TaskScheduler class to manage all task operations.
3. Implement a menu-driven interface for user interaction.
4. Store task data using CSV file handling.
5. Load existing tasks automatically when the application starts.
6. Generate unique Task IDs automatically.
7. Validate all user inputs (date, time, priority, status).
8. Implement CRUD operations (Create, Read, Update, Delete).
9. Implement task searching using Task ID or title.
10. Implement task sorting using Python sorting methods.
11. Implement task filtering based on status, priority, and date.
12. Implement reminder logic by comparing system time with task time.
13. Update task status automatically after completion.
14. Maintain a task history for completed tasks.
15. Handle file exceptions (missing or corrupted CSV files).
16. Handle invalid user input using exception handling.
17. Save changes automatically after every operation.
18. Display data in a clean tabular format.
19. Create reusable functions for each feature.
20. Use loops to keep the application running until the user exits.
21. Optimize the code using Object-Oriented Programming (OOP).
22. Add comments and documentation for maintainability.
23. Test all modules individually and together.
24. Debug and fix runtime and logical errors.
25. Ensure data consistency and prevent duplicate records.

Assign task to member

Mugdha Vinod Lad :

1. User can add a description for each task.
2. User can assign a category (Work, Study, Personal, etc.).
3. User can export tasks to a CSV file.
4. User can import tasks from a CSV file.

Unnati Thakur:

- System automatically saves tasks to prevent data loss.
- System validates date and time entered by the user.
- System prevents duplicate task IDs.
- User can clear completed tasks

Vaishnavi Vijay Teli :

21. User can set recurring tasks (Daily, Weekly, Monthly).
22. User can set multiple reminders for a task.
23. User can set reminder intervals (5, 10, 15, 30 minutes before).
24. User can choose notification sound or silent reminders.

Gayatri Shekhar Wadile :

- System records the task creation and completion timestamps.
- User can edit only specific fields without re-entering all task details.
- User can filter tasks by priority, date, category, or status.
- User can view tasks due today.
- User can view overdue tasks.

Krushna :

- User can add notes to tasks.
- System displays a confirmation message before deleting a task.
- User can exit the application safely after saving all data.

Bhoomi :

14. User can view task statistics (Total, Completed, Pending, Overdue).
15. User can archive completed tasks.
16. System automatically sorts upcoming tasks on startup.
17. User can back up and restore task data.
18. User can restore deleted tasks from a recycle bin (optional).